

PYTHON AUTOMATION – LAB ASSIGNMENTS

MODULE 2: UNIT TEST FRAMEWORKS

Assignments 7–9

Name: Swarnendu Boral

Roll No: 10

Section: CSE(AI)

Enrollment Number:-- 12023052020011

Overview

This document solves the three official Module 2 lab assignments listed in the syllabus: Assignment 7 (Page Object Model Restructure, sub-topic 2.3), Assignment 8 (Data-Driven Automation / DDT, sub-topic 2.1), and Assignment 9 (PyTest Integration with HTML Reporting, sub-topics 2.1/2.2). The three build on each other: Assignment 7 restructures the Module 1 login flow into a Page Object Model, Assignment 8 data-drives that same login flow, and Assignment 9 migrates it onto PyTest with automated HTML reporting.

Target application: <https://www.saucedemo.com/> (same site used for Assignment 1 in Module 1).

Assignment 7: Page Object Model (POM) Restructure

Module	Module 2 – Unit Test Frameworks (Lab Assignments)
Assignment Title	Page Object Model (POM) Restructure
Sub-topic Ref.	2.3 – Page Object Model

1. Problem Statement

The flat, script-style login test from Module 1 (Assignment 1) mixes locators, UI actions, and assertions in a single function. It must be restructured into a Page Object Model, where page classes hold only locators and UI methods, and the test file holds only assertions.

2. Objective

- Create a BasePage class with generic, reusable UI actions (click, type, get text).
- Create a LoginPage class that encapsulates the login form's locators and a login() action method.
- Create an InventoryPage class that encapsulates the post-login page's locators and a load-check method.
- Write a test class containing only page-object calls and assertions — no locators or raw Selenium calls.

3. Tools, Software & Concepts Used

- unittest (Python standard library)
- Page Object Model (POM) design pattern
- Practice site: <https://www.saucedemo.com/>
- Concepts: separation of locators/UI-actions from test assertions

4. Implementation Steps

1. Create pages/base_page.py with generic click(), type_text(), and get_text() helper methods.
2. Create pages/login_page.py (subclassing BasePage) with locators for username, password, login button, and error banner, plus a login(username, password) method and get_error_text() method.
3. Create pages/inventory_page.py (subclassing BasePage) with a locator for the inventory container and an is_loaded() method.
4. Create test_login_pom.py using unittest.TestCase with setUp/tearDown for driver lifecycle, and one test method that only calls page-object methods and asserts the result.
5. Run with: `python -m unittest test_login_pom.py`

5. Source Code

```
# ----- pages/base_page.py -----
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class BasePage:
```

```

"""Generic UI actions shared by every page object. Holds no test
assertions."""

def __init__(self, driver, timeout=10):
    self.driver = driver
    self.wait = WebDriverWait(driver, timeout)

def click(self, locator):
    self.wait.until(EC.element_to_be_clickable(locator)).click()

def type_text(self, locator, text):
    element = self.wait.until(EC.visibility_of_element_located(locator))
    element.clear()
    element.send_keys(text)

def get_text(self, locator):
    return self.wait.until(EC.visibility_of_element_located(locator)).text

def is_visible(self, locator):
    try:
        return
        self.wait.until(EC.visibility_of_element_located(locator)).is_displayed()
    except Exception:
        return False

# ----- pages/login_page.py -----
from selenium.webdriver.common.by import By
from pages.base_page import BasePage

class LoginPage(BasePage):
    USERNAME_INPUT = (By.ID, 'user-name')
    PASSWORD_INPUT = (By.NAME, 'password')
    LOGIN_BUTTON = (By.XPATH, "//input[@id='login-button']")

```

```

ERROR_BANNER = (By.CSS_SELECTOR, "h3[data-test='error']")

URL = 'https://www.saucedemo.com/'

def open(self):
    self.driver.get(self.URL)

def login(self, username, password):
    self.type_text(self.USERNAME_INPUT, username)
    self.type_text(self.PASSWORD_INPUT, password)
    self.click(self.LOGIN_BUTTON)

def get_error_text(self):
    return self.get_text(self.ERROR_BANNER)

# ----- pages/inventory_page.py -----
from selenium.webdriver.common.by import By
from pages.base_page import BasePage

class InventoryPage(BasePage):
    INVENTORY_CONTAINER = (By.ID, 'inventory_container')

    def is_loaded(self):
        return self.is_visible(self.INVENTORY_CONTAINER)

# ----- test_login_pom.py -----
import unittest
from selenium import webdriver
from pages.login_page import LoginPage
from pages.inventory_page import InventoryPage

```

```

class TestLoginPOM(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()

    def tearDown(self):
        self.driver.quit()

    def test_valid_login_reaches_inventory_page(self):
        # NOTE: this test method contains ONLY page-object calls and an
        # assertion -
        # no locators, no raw find_element calls.
        login_page = LoginPage(self.driver)
        login_page.open()
        login_page.login('standard_user', 'secret_sauce')

        inventory_page = InventoryPage(self.driver)
        loaded = inventory_page.is_loaded()

        self.driver.save_screenshot('picA7_inventory_loaded.png') # automatic
        # evidence capture
        self.assertTrue(loaded, 'Inventory page did not load after a valid
        login')

if __name__ == '__main__':
    unittest.main()

```

6. Output

Running the test module logs in through the LoginPage object, verifies the InventoryPage loaded through the InventoryPage object, and automatically saves a screenshot as evidence right after the assertion check — with no locators anywhere inside the test file itself.

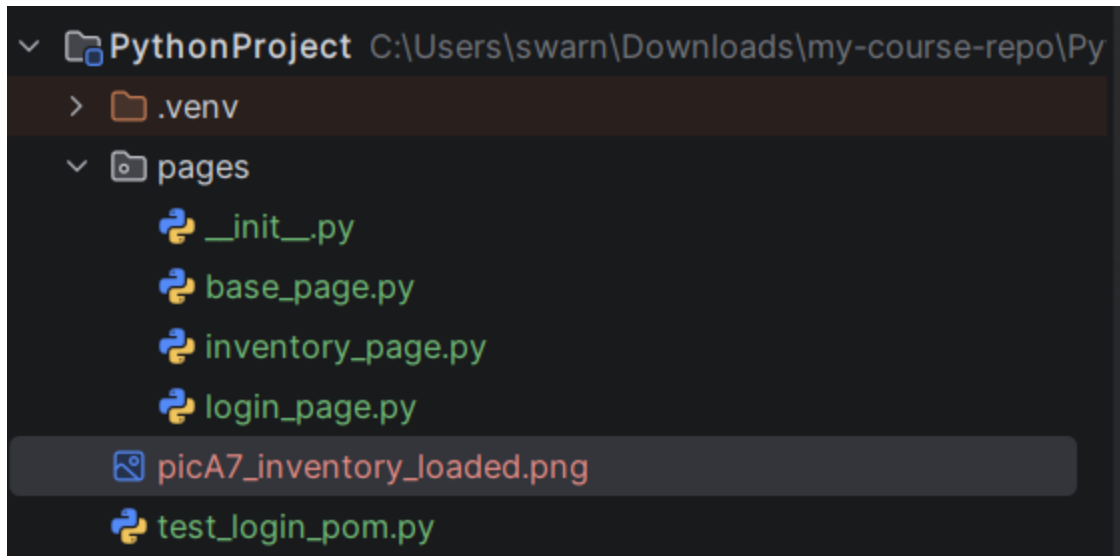


Fig A7.1 — Project file structure showing `pages/base_page.py`, `login_page.py`, `inventory_page.py`, and `test_login_pom.py`.

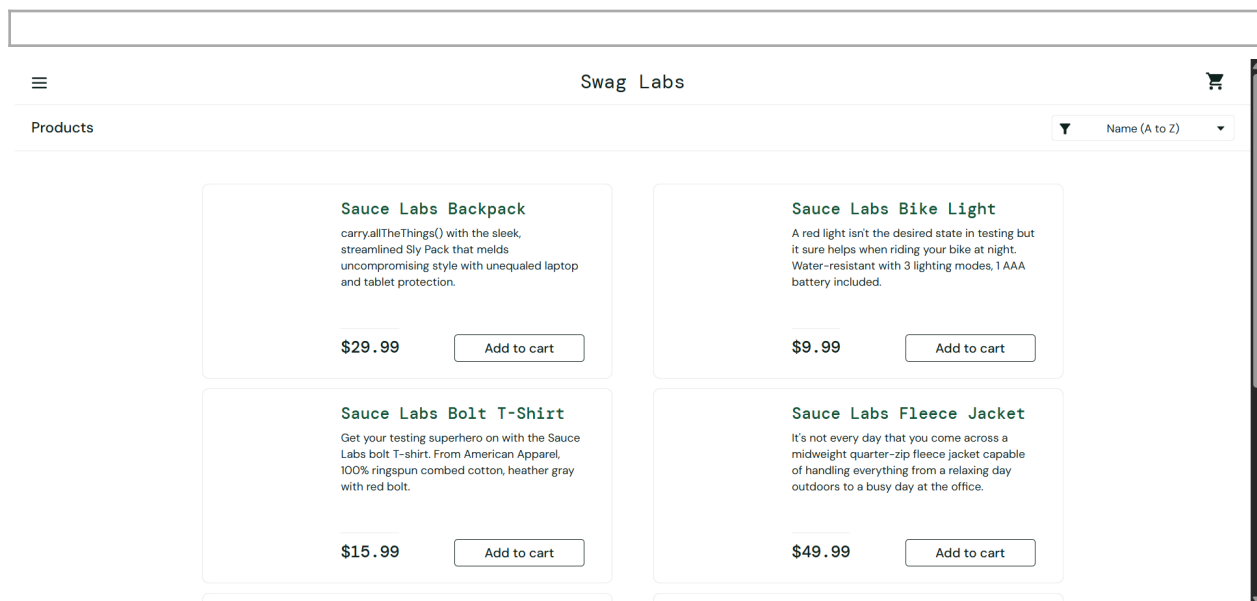


Fig A7.2 — `inventory_loaded.png` (auto-captured) showing the SauceDemo inventory page after login.

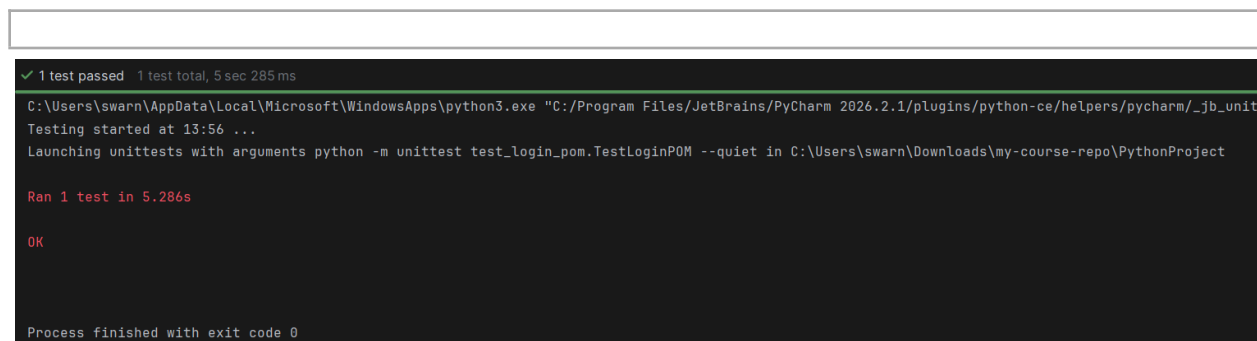


Fig A7.3 — Terminal output showing the unittest run result ('OK', 1 test passed).

7. Result

The Module 1 login flow was successfully restructured into a Page Object Model: BasePage, LoginPage, and InventoryPage each hold only locators/UI actions, while the test class holds only the assertion.

8. Observation

- Keeping login() and get_error_text() inside LoginPage (rather than in the test) means that if SauceDemo ever changes its input ids, only login_page.py needs to change — every test that logs in stays untouched.
- BasePage centralizing WebDriverWait means every page object automatically waits correctly without each page class re-implementing its own timing logic.
- The test method reads almost like plain English (open → login → check inventory loaded) specifically because all Selenium mechanics are hidden behind the page objects — that readability is the main payoff of POM, not just code reuse.

9. Conclusion

A flat script was successfully converted into a maintainable Page Object Model structure, cleanly separating locators and UI actions from test logic — the foundation the next two assignments build on.

Assignment 8: Data-Driven Automation (DDT)

Module	Module 2 – Unit Test Frameworks (Lab Assignments)
Assignment Title	Data-Driven Automation (DDT)
Sub-topic Ref.	2.1 – Unittest (Data-Driven Testing)

1. Problem Statement

A login script must be validated against multiple username/password combinations — valid, invalid, and locked-out — without hardcoding each case, and must assert that the correct outcome (success or the correct error message) shows up for every combination.

2. Objective

- Store multiple login test cases (username, password, expected outcome) in an external CSV file.
- Loop through the CSV data inside a single unittest test method using subTest() so all rows run and report independently, even if one fails.
- Assert the correct outcome — successful navigation to the inventory page, or the correct error message — for each row.

3. Tools, Software & Concepts Used

- unittest.TestCase.subTest()
- csv (Python standard library)
- Reused LoginPage / InventoryPage page objects from Assignment 7
- Practice site: <https://www.saucedemo.com/>

4. Implementation Steps

1. Create data/login_test_data.csv with columns: username, password, expected.
2. Include at least one valid case (expected='success'), one invalid-credentials case (expected='invalid_credentials'), and one locked-out case (expected='locked_out').
3. In the test method, read the CSV into a list of dicts.
4. Loop through each row inside self.subTest(username=row['username']): perform the login, then assert the correct outcome for that row's 'expected' value.
5. After each login attempt, automatically save a per-row screenshot as evidence before asserting.

5. Source Code

```
# ----- data/login_test_data.csv -----
# username,password,expected
# standard_user,secret_sauce,success
# standard_user,wrong_password,invalid_credentials
# locked_out_user,secret_sauce,locked_out
# not_a_real_user,secret_sauce,invalid_credentials

# ----- data/test_login_ddt.py -----
import csv
import os
import unittest
```

```

from selenium import webdriver
from pages.login_page import LoginPage
from pages.inventory_page import InventoryPage

DATA_FILE = os.path.join('data', 'login_test_data.csv')

def load_login_data():
    with open(DATA_FILE, newline='', encoding='utf-8') as f:
        return list(csv.DictReader(f))

class TestLoginDDT(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.login_page = LoginPage(self.driver)
        self.inventory_page = InventoryPage(self.driver)

    def tearDown(self):
        self.driver.quit()

    def test_login_combinations(self):
        test_cases = load_login_data()

        for row in test_cases:
            with self.subTest(username=row['username'],
expected=row['expected']):
                self.login_page.open()
                self.login_page.login(row['username'], row['password'])

                # Automatic screenshot for every row, taken BEFORE the assertion
so
                # evidence exists regardless of whether this row passes or
fails.

```

```

        safe_name = row['username'].replace(' ', '_')
        self.driver.save_screenshot(f'picA8_{safe_name}_result.png')

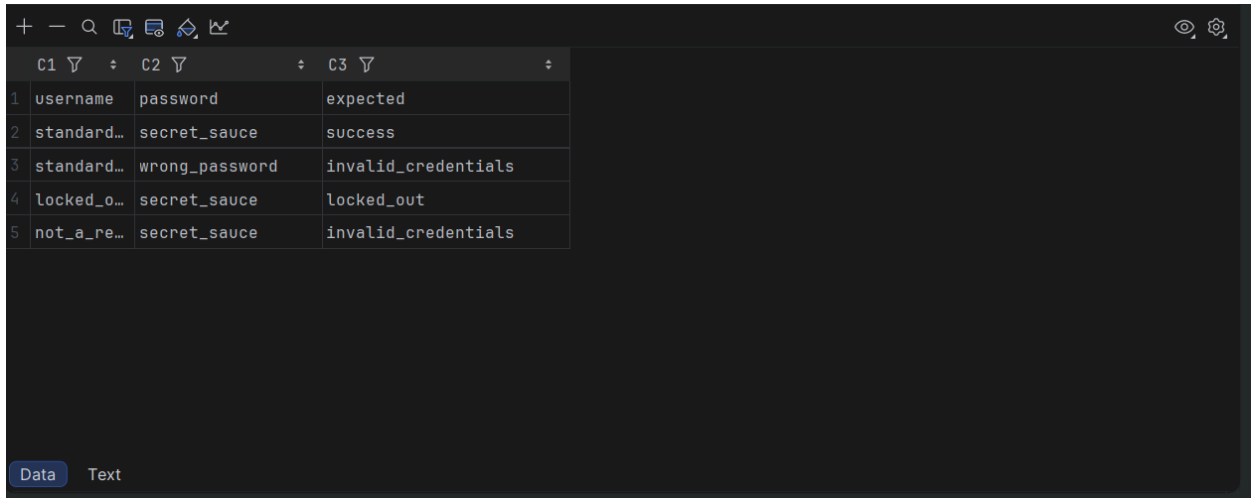
        if row['expected'] == 'success':
            self.assertTrue(
                self.inventory_page.is_loaded(),
                f"Expected successful login for '{row['username']}'"
            )
        elif row['expected'] == 'locked_out':
            self.assertIn('locked out',
                self.login_page.get_error_text().lower())
        elif row['expected'] == 'invalid_credentials':
            self.assertIn('do not match',
                self.login_page.get_error_text().lower())
        else:
            self.fail(f"Unknown expected outcome: {row['expected']}")

if __name__ == '__main__':
    unittest.main()

```

6. Output

The script runs all four CSV-driven login combinations inside one test method, automatically saving a screenshot for each row before checking the correct success/error outcome, and unittest reports each subTest independently in the terminal summary.



	C1	C2	C3
1	username	password	expected
2	standard...	secret_sauce	success
3	standard...	wrong_password	invalid_credentials
4	locked_o...	secret_sauce	locked_out
5	not_a_re...	secret_sauce	invalid_credentials

Fig A8.1 — login_test_data.csv opened in a text editor / Excel showing the four test rows.

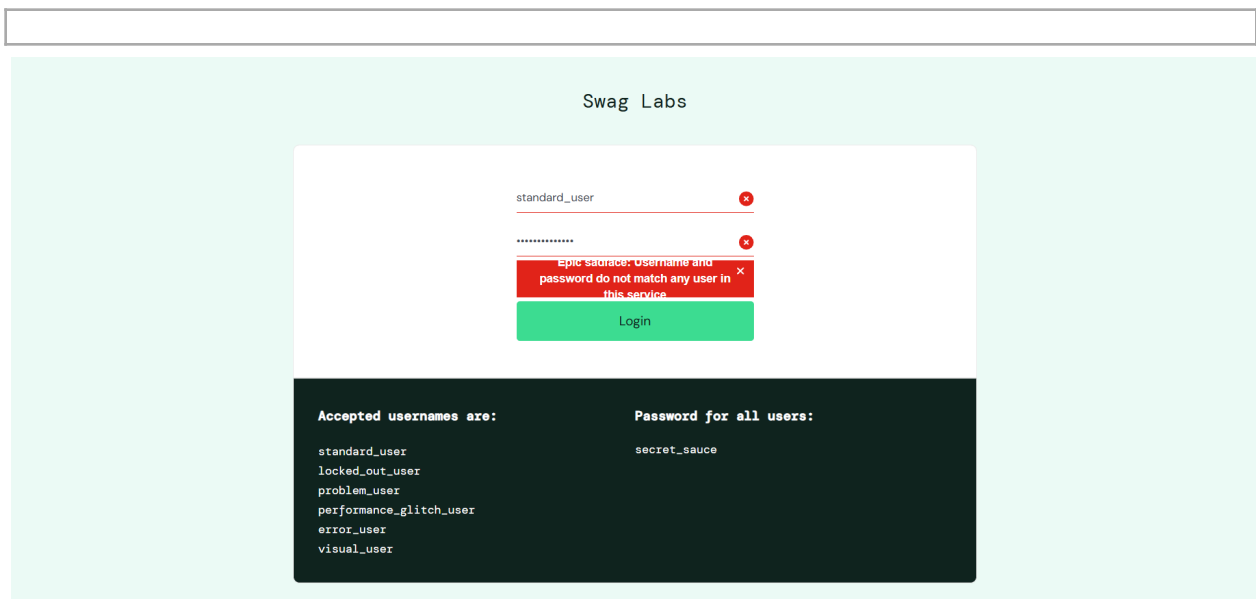


Fig A8.2 — standard_user_result.png showing the inventory page for the valid-login row.

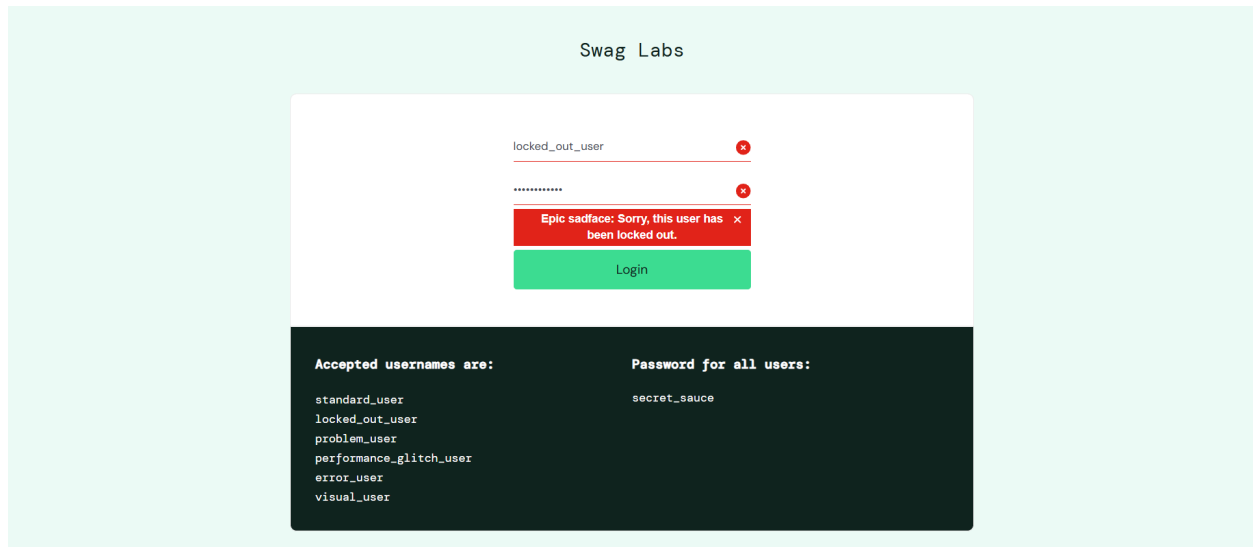


Fig A8.3 — `locked_out_user_result.png` showing the 'locked out' error banner.

7. Result

All four username/password combinations were exercised from external CSV data, with the correct success/error outcome verified and screenshot evidence captured automatically for every row.

8. Observation

- `subTest()` lets all rows execute in a single test run even if one row's assertion fails — without it, the first failing row would stop the entire test method and hide results for every row after it.
- Capturing the screenshot before the assertion (rather than only on failure) guarantees evidence exists for every row, including passing ones — useful for a lab record where each case needs to be shown, not just failures.
- Separating 'expected outcome' as data (a CSV column) rather than a hardcoded if/else per username is what makes this genuinely data-driven — adding a fifth test case means adding a CSV row, not touching the script.

9. Conclusion

A single, data-driven test method replaced what would otherwise be four near-duplicate hardcoded test methods, demonstrating the core value of DDT: scaling test coverage by adding data, not code.

Assignment 9: PyTest Integration with HTML Reporting

Module	Module 2 – Unit Test Frameworks (Lab Assignments)
Assignment Title	PyTest Integration with HTML Reporting
Sub-topic Ref.	2.1 / 2.2 – Unittest to PyTest Migration + HTML Reporting

1. Problem Statement

The Unittest-based login suite must be migrated onto PyTest, using fixtures for browser setup/teardown instead of setUp/tearDown, and the suite must automatically produce an HTML execution report with screenshots embedded directly into any failed test's entry.

2. Objective

- Replace unittest's setUp/tearDown with a PyTest driver fixture.
- Rewrite the login assertions as plain PyTest test functions using that fixture.
- Configure pytest-html so running the suite from the terminal auto-generates a self-contained HTML report.
- Automatically embed a screenshot into the HTML report for any test that fails, with no manual step per test.

3. Tools, Software & Concepts Used

- pytest, pytest-html (pip install pytest pytest-html)
- PyTest fixtures, conftest.py, pytest_runtest_makereport hook
- Reused LoginPage / InventoryPage page objects from Assignment 7

4. Implementation Steps

1. Install the required packages: pip install pytest pytest-html
2. In conftest.py, create a `driver` fixture that launches Chrome and yields it, quitting it after each test.
3. In conftest.py, implement a pytest_runtest_makereport hookwrapper that — only when a test's call phase fails — automatically saves a screenshot and attaches it to the HTML report via pytest_html's extras.image().
4. In test_login_pytest.py, rewrite the login assertions as plain functions that accept the `driver` fixture.
5. Run: pytest --html=report.html --self-contained-html -v
6. Open report.html and confirm failed rows show an embedded screenshot with no manual capture required.

5. Source Code

```
# ----- conftest.py -----
import os
import base64
import pytest
from pytest_html import extras
from selenium import webdriver
```

```

@pytest.fixture
def driver():
    drv = webdriver.Chrome()
    drv.maximize_window()
    yield drv
    drv.quit()

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    extra = getattr(report, 'extras', []) # renamed from 'extra' in
    pytest-html 4.x

    if report.when == 'call' and report.failed:
        driver = item.funcargs.get('driver')
        if driver is not None:
            os.makedirs('screenshots', exist_ok=True)
            screenshot_path = f'screenshots/{item.name}.png'
            driver.save_screenshot(screenshot_path)

            # Base64-encode so the image is actually EMBEDDED in the html
            # file, not just linked - this is what --self-contained-html
            # is supposed to guarantee.
            with open(screenshot_path, 'rb') as f:
                encoded = base64.b64encode(f.read()).decode('utf-8')
            extra.append(extras.image(encoded, name='Failure Screenshot'))

    report.extras = extra # renamed from 'extra' in pytest-html 4.x

```

```

# ----- test_login_pytest.py -----

from pages.login_page import LoginPage
from pages.inventory_page import InventoryPage

def test_valid_login(driver):
    login_page = LoginPage(driver)
    login_page.open()
    login_page.login('standard_user', 'secret_sauce')

    inventory_page = InventoryPage(driver)
    assert inventory_page.is_loaded()

```

```

def test_invalid_login_shows_error(driver):
    login_page = LoginPage(driver)
    login_page.open()
    login_page.login('standard_user', 'wrong_password')
    assert 'do not match' in login_page.get_error_text().lower()

def test_locked_out_user_shows_error(driver):
    login_page = LoginPage(driver)
    login_page.open()
    login_page.login('locked_out_user', 'secret_sauce')
    assert 'locked out' in login_page.get_error_text().lower()

def test_deliberately_wrong_assertion_to_prove_reporting(driver):
    # Intentionally wrong assertion so the report has a genuine FAILED row
    # with an embedded screenshot to inspect (remove once report is verified).
    login_page = LoginPage(driver)
    login_page.open()
    login_page.login('standard_user', 'secret_sauce')

    inventory_page = InventoryPage(driver)
    assert not inventory_page.is_loaded(), 'Deliberate failure to verify HTML
report + screenshot embedding'

```

6. Output

Running `pytest --html=report.html --self-contained-html -v` executes all four tests using the driver fixture, and opening report.html afterward shows three PASSED rows and one FAILED row — with a screenshot automatically embedded directly inside the failed row's expandable detail section.




```

st_login_pytest.py:39: AssertionError
..... Generated html report: file:///C:/Users/swann/Downloads/my-course-repo/PythonProject/report.html .....
===== short test summary info =====
FAILED test_login_pytest.py::test_deliberately_wrong_assertion_to_prove_reporting - AssertionError: Deliberate failure to verify HTML report + screenshot embedding
===== 1 failed, 5 passed, 4 subtests passed in 31.44s =====

```

Fig A9.1 — Terminal output of the pytest run showing 5 passed, 1 failed.



Fig A9.2 — report.html opened in a browser, summary table showing all four test results.



Fig A9.3 — The failed row expanded in report.html, showing the embedded screenshot inline.

7. Result

The suite was fully migrated from unittest to PyTest fixtures, and the HTML report correctly embedded a screenshot for the failing test automatically, with zero manual screenshot steps.

8. Observation

- A PyTest fixture with `yield` cleanly separates setup (before yield) from teardown (after yield) in a single function, replacing the two separate setUp()/tearDown() methods unittest required.
- The pytest_runtest_makereport hook must check report.when == 'call' specifically — checking on the 'setup' or 'teardown' phase instead would either miss real test failures or misfire on fixture errors.
- The last test is deliberately written to fail — this is intentional and necessary to prove the screenshot embedding actually works; without at least one real failure, the HTML report's screenshot logic is never exercised at all. Remove or fix that test once verified.

9. Conclusion

The login suite now runs on PyTest with fixture-based lifecycle management and produces a self-contained HTML report that automatically embeds failure screenshots, completing the Module 2 progression from raw script → POM → data-driven Unittest → fixture-based PyTest with reporting.